



Instituto Tecnológico de Mazatlán



**Comparación de delay() y millis()
con Arduino UNO R4 WiFi**

Carrera:

Ingeniería en Sistemas Computacionales

Materia:

Sistemas Programables

Profesor:

Miguel Barrón Hernández

Integrantes:

Ximena Armenta Dávila

Carlos Tadeo Ibarra Castañeda

Dulce Princesa Ulivarría González

1. Introducción

Uno de los conceptos fundamentales al programar un microcontrolador es la diferencia entre una espera bloqueante y una no bloqueante. La función `delay()` detiene por completo la ejecución del programa durante el tiempo indicado, mientras que `millis()` permite medir el tiempo transcurrido sin interrumpir el resto del código, lo que hace posible controlar varios eventos de manera independiente y simultánea.

En la presente práctica se compararon ambos enfoques utilizando un Arduino UNO R4 WiFi y tres LEDs (verde, amarillo y rojo), cada uno con un intervalo de parpadeo distinto (500 ms, 1000 ms y 1500 ms, respectivamente). Se implementaron dos programas independientes, uno con `delay()` y otro con `millis()`, manteniendo el mismo circuito y los mismos intervalos, con el fin de observar y documentar las diferencias en el comportamiento de los LEDs.

2. Objetivos

Objetivo general:

- Comparar el comportamiento de un mismo circuito de tres LEDs controlado mediante `delay()` y mediante `millis()` en un Arduino UNO R4 WiFi, evidenciando las diferencias entre una espera bloqueante y una no bloqueante.

Objetivos específicos:

- Implementar un programa con `delay()` que controle tres LEDs con distintos intervalos de tiempo.
- Implementar un programa equivalente con `millis()` que controle los mismos LEDs con los mismos intervalos, sin bloquear la ejecución.
- Enviar un mensaje por el monitor serie cada vez que se enciende el LED rojo, únicamente en la versión con `millis()`.
- Verificar en el armado físico que ambas versiones producen el mismo circuito, pero con un comportamiento distinto en el parpadeo de los LEDs.

3. Herramientas y material utilizado

- Arduino UNO R4 WiFi
- 3 LEDs (1 verde, 1 amarillo y 1 rojo)
- 3 resistencias de 330 Ω
- Protoboard y cables de conexión (jumpers)
- Arduino IDE 2 (con monitor serie a 9600 baudios)

4. Desarrollo / Metodología

El circuito es el mismo para ambas versiones del programa: el LED verde se conectó al pin digital 9, el amarillo al pin 10 y el rojo al pin 11, cada uno a través de su resistencia limitadora de 330 Ω hacia GND.

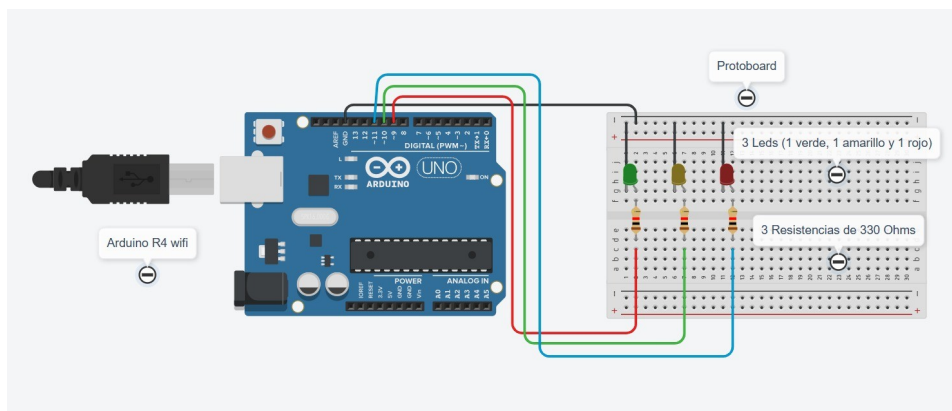


Figura 1. Diagrama esquemático del circuito: Arduino UNO R4 WiFi, protoboard, 3 LEDs y 3 resistencias de 330 Ω .

Se programaron dos versiones (Codigo/Delay/Delay.ino y Codigo/Millis/millis.ino) que ejecutan la misma lógica de intervalos, pero con estrategias de temporización distintas. La versión con delay() enciende y apaga cada LED de forma secuencial, deteniendo por completo el programa mientras espera cada intervalo; por ello, en todo momento hay como máximo un LED encendido. La versión con millis() calcula el tiempo transcurrido en cada iteración del loop() y actualiza cada LED de forma independiente en cuanto se cumple su propio intervalo, sin esperar a los demás, lo que permite que varios LEDs estén encendidos al mismo tiempo. Además, solo la versión con millis() envía un mensaje por el puerto serie (a 9600 baudios) cada vez que enciende el LED rojo; la versión con delay() no utiliza el monitor serie.

Solo la versión con millis() envía datos por el monitor serie (mensaje al encender el LED rojo).
La versión con delay() usa el mismo circuito y las mismas salidas, pero no imprime nada en el monitor serie.

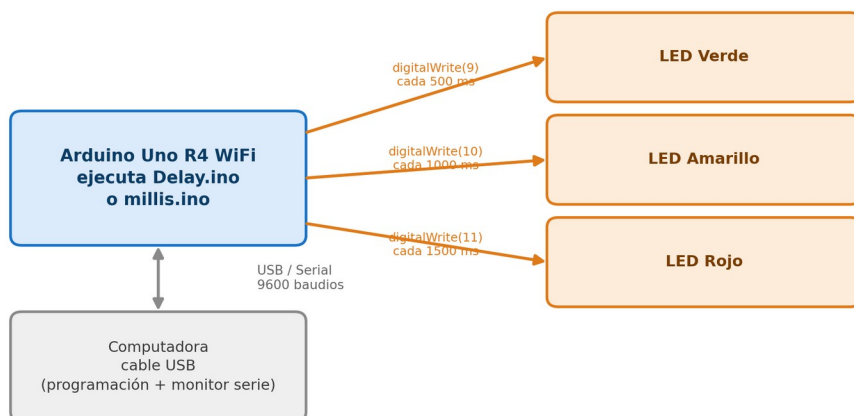


Figura 2. Diagrama de bloques del sistema: el Arduino ejecuta Delay.ino o millis.ino y controla los LEDs verde, amarillo y rojo; solo millis.ino envía datos por el monitor serie.

5. Evidencia del circuito y del monitor serie funcionando

A continuación se presenta la captura del monitor serie del Arduino IDE 2, ejecutando millis.ino, donde se observa el mensaje "Ximena la mas chambeadora" repetido cada vez que se enciende el LED rojo (cada 3000 ms). La versión con delay() no imprime nada en el monitor serie.

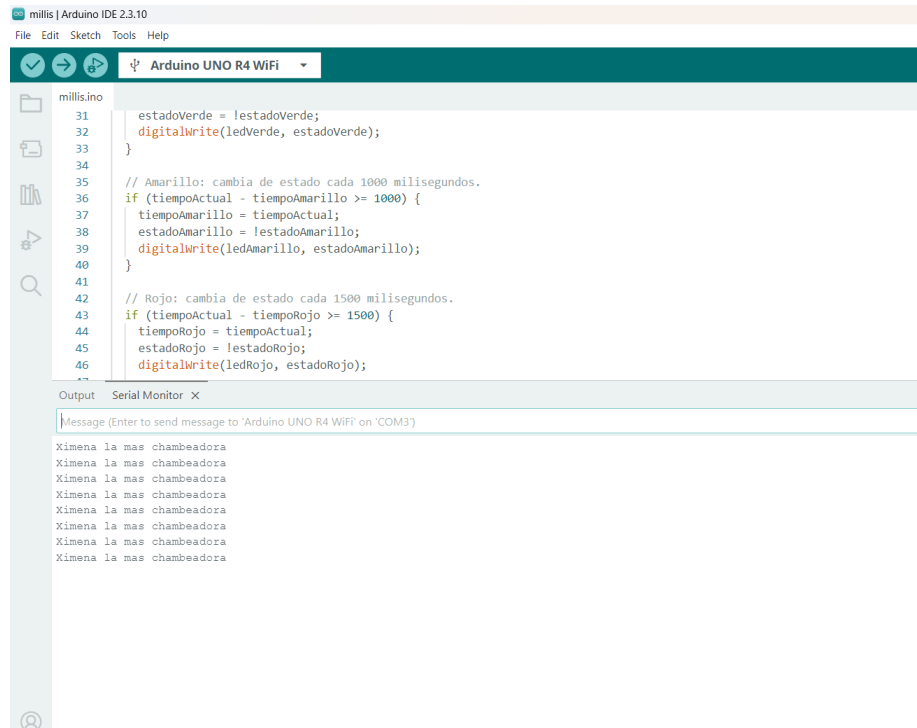


Figura 3. Monitor serie del Arduino IDE 2 ejecutando millis.ino: el mensaje se repite cada vez que se enciende el LED rojo.

Asimismo, se presenta la evidencia fotográfica del circuito físico armado en protoboard, común a ambas versiones del programa, durante las pruebas de la práctica.

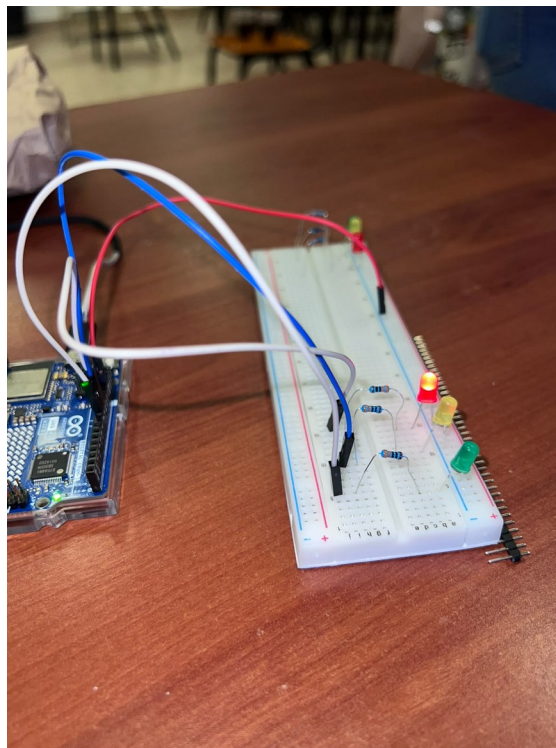


Figura 4. Circuito armado en protoboard: LEDs verde, amarillo y rojo con sus resistencias de 330 Ω .

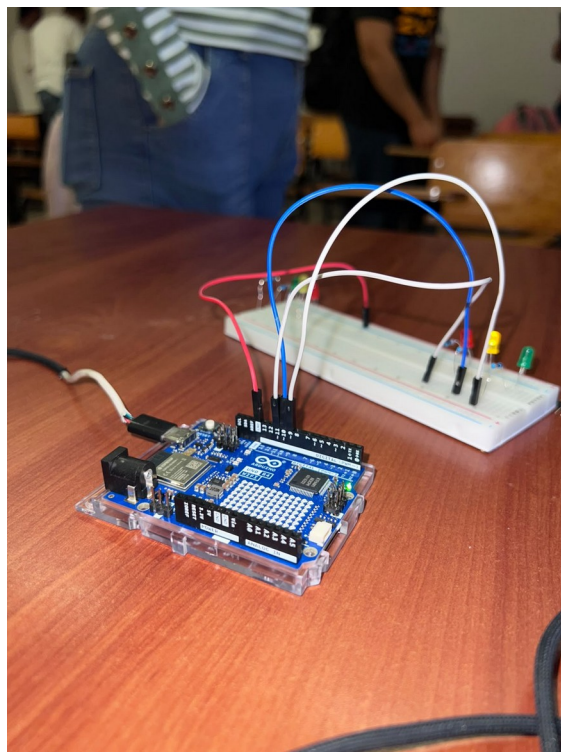


Figura 5. Conexión del protoboard al Arduino UNO R4 WiFi durante las pruebas.

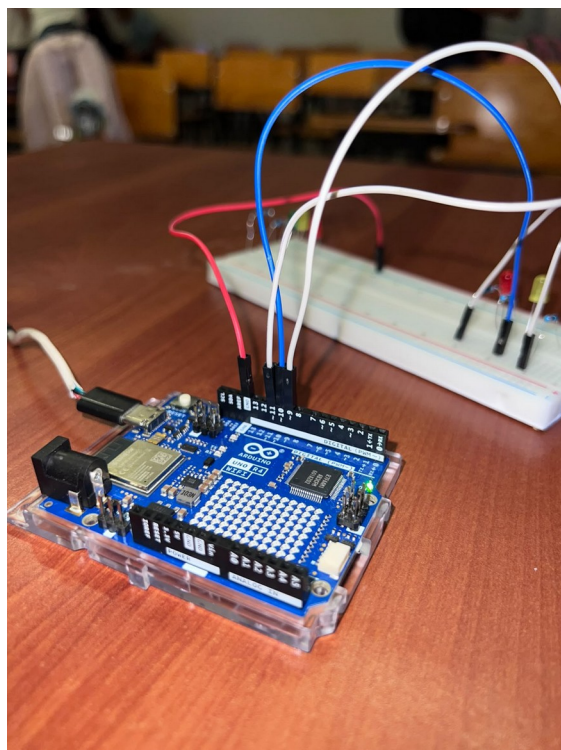


Figura 6. Detalle de las conexiones en los pines digitales del Arduino UNO R4 WiFi.

6. Análisis de resultados

Las pruebas se realizaron el 15 de septiembre de 2026, comparando los dos programas que controlan los mismos tres LEDs con los mismos intervalos de tiempo. El comportamiento de cada versión se determinó a partir del código fuente y se verificó en el armado físico, registrado en los videos de la práctica.

Ambas versiones controlan tres LEDs con los mismos intervalos (verde cada 500 ms, amarillo cada 1000 ms y rojo cada 1500 ms), pero difieren en cómo manejan el tiempo. Las Figuras 7 y 8 muestran el estado de cada LED durante una ventana de 3000 ms, calculado directamente a partir de la lógica de cada programa.

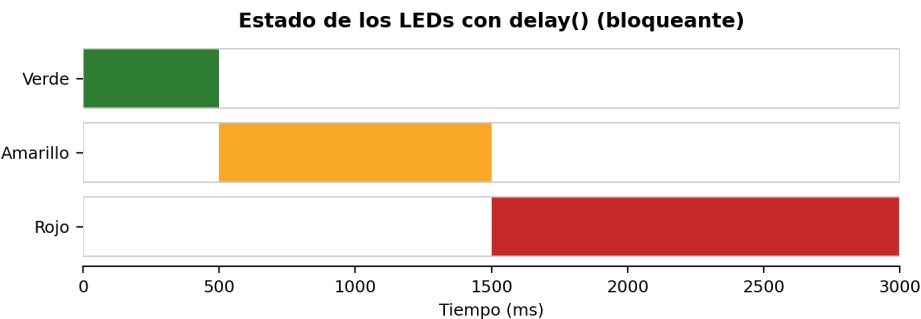


Figura 7. Estado de los LEDs con delay(): se encienden uno a la vez, nunca al mismo tiempo, porque el programa detiene por completo la ejecución mientras espera cada uno.

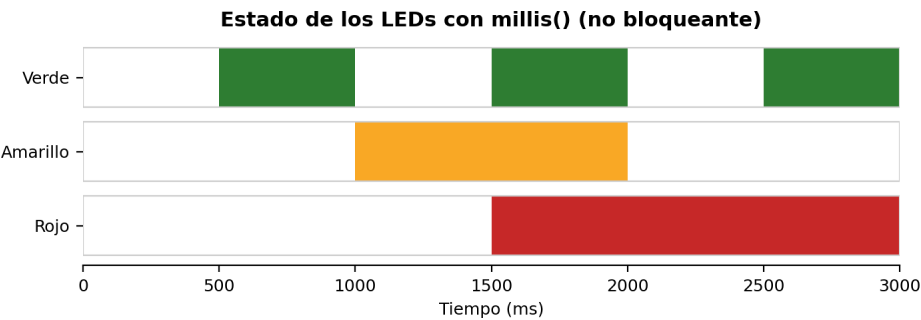


Figura 8. Estado de los LEDs con millis(): cada LED cambia de estado en su propio intervalo sin esperar a los demás, por lo que pueden estar encendidos varios al mismo tiempo (por ejemplo, verde y rojo entre 2500 y 3000 ms).

La Tabla I resume los intervalos configurados en el código para cada LED. La Tabla II detalla, para cada versión del programa, los intervalos en los que cada LED permanece encendido dentro de la ventana de 0 a 3000 ms mostrada en las Figuras 7 y 8.

Tabla I. Intervalos configurados por LED.

LED	Pin	Intervalo	Comportamiento
Verde	9	500 ms	Cambia de estado cada 500 ms (millis) / permanece encendido 500 ms por ciclo (delay)
Amarillo	10	1000 ms	Cambia de estado cada 1000 ms (millis) / permanece encendido 1000 ms por ciclo (delay)

LED	Pin	Intervalo	Comportamiento
Rojo	11	1500 ms	Cambia de estado cada 1500 ms (millis, y envía mensaje al monitor serie al encender) / permanece encendido 1500 ms por ciclo (delay)

Tabla II. Intervalos de encendido de cada LED, por versión del programa.

Versión	LED	Encendido dentro de 0–3000 ms
delay()	Verde	0–500 ms
delay()	Amarillo	500–1500 ms
delay()	Rojo	1500–3000 ms
millis()	Verde	500–1000, 1500–2000, 2500–3000 ms
millis()	Amarillo	1000–2000 ms
millis()	Rojo	1500–3000 ms

Observaciones sobre el comportamiento del sistema:

- Con delay(), en cualquier instante hay como máximo un LED encendido; el ciclo completo dura exactamente 3000 ms (500 + 1000 + 1500) y se repite indefinidamente en ese orden.
- Con millis(), cada LED respeta su propio intervalo sin esperar a los demás; en la ventana analizada, verde y rojo coinciden encendidos entre 2500 y 3000 ms, algo que la versión con delay() nunca permite.
- El mensaje «Ximena la mas chambeadora» solo se envía por el monitor serie en la versión con millis(), exactamente cuando el LED rojo se enciende (en $t = 1500$ ms y cada 3000 ms después); la versión con delay() no usa el monitor serie.
- El armado físico (el mismo circuito para ambas versiones) y los videos de la práctica confirman visualmente este comportamiento: con delay() los LEDs parpadean en secuencia, uno a la vez, y con millis() lo hacen de forma independiente.
- Los tiempos mostrados se derivan directamente de los intervalos definidos en el código (500, 1000 y 1500 ms); no se realizaron mediciones de precisión del microcontrolador ni de posible retraso acumulado por el tiempo de ejecución del propio loop().

7. Conclusiones individuales

Ximena

Esta práctica me permitió comprobar de forma muy clara por qué delay() no es recomendable cuando se necesita controlar varios eventos a la vez: al usarlo, los tres LEDs quedan obligados a turnarse, aunque en el código cada uno tenga su propio intervalo. Ver la gráfica de delay() y notar que nunca hay dos LEDs encendidos al mismo tiempo hizo evidente el costo de bloquear la ejecución del programa.

También me pareció interesante trabajar con el monitor serie únicamente en la versión con millis(), ya que reforzó la idea de que, al no bloquear el programa con delay(), es posible aprovechar el tiempo de ejecución para otras tareas, como enviar mensajes de depuración, sin afectar el parpadeo de los LEDs. Ver el mensaje repetirse exactamente cada vez que el LED rojo se encendía fue una buena forma de confirmar que la lógica de tiempos estaba funcionando correctamente.

Carlos Tadeo

El desarrollo de esta práctica reforzó mi comprensión sobre el uso de `millis()` para manejar múltiples temporizadores independientes dentro de un mismo `loop()`, calculando el tiempo transcurrido en lugar de detener el programa. Implementar ambas versiones con el mismo circuito me permitió aislar la variable que realmente estábamos comparando: la estrategia de temporización, y no el hardware.

Un punto que me pareció clave fue notar que, aunque el código de `millis()` es un poco más largo que el de `delay()`, la lógica de comparar `tiempoActual - tiempoAnterior` contra el intervalo deseado es bastante sencilla una vez que se entiende el patrón, y se puede repetir para tantos LEDs o eventos como se necesiten sin que interfieran entre sí. Esto me hizo valorar `millis()` como una herramienta mucho más escalable para proyectos con varios componentes.

Dulce Princesa

Considero que el mayor aprendizaje de esta práctica fue entender la diferencia entre bloquear y no bloquear la ejecución de un programa, algo que hasta ahora solo había visto de forma teórica. Comparar las Tablas I y II me ayudó a visualizar con números concretos cómo el mismo conjunto de intervalos produce resultados completamente distintos según la estrategia de temporización utilizada.

También me quedó claro que estas diferencias no son solo teóricas: en un proyecto real, usar `delay()` podría impedir que el Arduino atendiera un botón, un sensor o una comunicación mientras espera un LED, mientras que `millis()` permite que todo el sistema siga respondiendo. Ver esto reflejado tanto en las gráficas como en el circuito físico armado me dejó una base sólida para decidir, en futuras prácticas, cuál de las dos estrategias conviene usar según lo que el proyecto necesite.

Referencias

[1] Arduino, "Arduino UNO R4 WiFi", documentación oficial. [En línea]. Disponible en: <https://docs.arduino.cc>

[2] Arduino, "Blink Without Delay", ejemplo oficial de temporización no bloqueante con `millis()`. [En línea]. Disponible en: <https://docs.arduino.cc/built-in-examples/digital/BlinkWithoutDelay>